Ensemble Learning — Theory Concepts

Smart Outcome Predictor · Part A: Conceptual Foundation · Supervised Learning

1. Ensemble Learning Methods

Definition. An **ensemble** combines the predictions of several models (*base learners* or *weak learners*) to produce one final prediction that is, on average, more accurate and more stable than any individual model.

Why they work. (a) **Statistical:** many different hypotheses fit the training data equally well; averaging them reduces the risk of choosing a poor one. (b) **Computational:** local-search learners (trees, neural networks) get stuck in different optima; combining them approximates the best one. (c) **Representational:** a weighted combination can represent functions that no single base learner can.

For an average of B models with individual variance σ^2 and pairwise correlation ρ, the variance of the average is:

$$Var(average) = \rho \cdot \sigma^2 + (1 - \rho) \cdot \sigma^2 / B$$

Hence ensembles help most when members are **accurate** and **diverse** (low ρ). Diversity is created by resampling the data (bagging), re-weighting it (boosting), using different algorithms (voting/stacking) or random feature subsets.

2. Bagging vs Boosting vs Voting

	Bagging	Boosting	Voting
Training scheme	Parallel; each model sees a bootstrap sample (≈ 63.2 % unique rows)	Sequential; model t+1 focuses on errors of models 1…t	Parallel; every model sees the same data
Base learners	Same algorithm, usually deep (high-variance) trees	Same algorithm, usually shallow (high-bias) trees	Different algorithms (LR, KNN, tree, SVM…)
Combination	Majority vote / mean	Weighted sum of learners	Hard: majority of labels. Soft: mean of probabilities
Main benefit	Lower **variance**	Lower **bias** (and some variance)	Cancels out diverse errors
Risk	Little effect on bias; correlated trees	Over-fits noisy labels / outliers	Weak or correlated members add little
Examples	Bagging, Random Forest	AdaBoost, Gradient Boosting, XGBoost	VotingClassifier / VotingRegressor

Out-of-bag (OOB) estimate. Each bootstrap sample leaves out ≈ 36.8 % of the rows; predicting those rows with the trees that never saw them gives a free, nearly unbiased validation score for bagging.

3. Bias–Variance Trade-off

For squared error, the expected prediction error at a point decomposes as:

$$E[(y - \hat{y})^2] = Bias^2 + Variance + \sigma^2(noise)$$

- **Bias** – error from wrong/too-simple assumptions (under-fitting).
- **Variance** – sensitivity to the particular training sample (over-fitting).
- **Irreducible noise σ^2** – cannot be removed by any model; it sets the ceiling on accuracy.

How ensembles address it. Bagging keeps the low bias of deep trees and shrinks variance by averaging. **Boosting** starts with high-bias weak learners and reduces bias by adding learners that fit the remaining error (shrinkage and sub-sampling keep variance in check). **Voting** and **stacking** blend heterogeneous models so that their individual errors partly offset each other.

4. Voting Classifier and Stacking Ensemble

Voting Classifier. Independent classifiers are trained on the same data and their outputs are merged by a fixed rule.

- **Hard voting:** predicted class = the class receiving most votes. Uses only labels, so no probability ranking (no meaningful ROC-AUC).
- **Soft voting:** predicted class = argmax of the (weighted) average of predicted class probabilities. Uses model confidence and usually performs at least as well when probabilities are reasonably calibrated.

Stacking (stacked generalisation). Level-0 base models are trained with K-fold cross-validation; their *out-of-fold* predictions form a new feature matrix on which a **meta-learner** (level-1 model such as Logistic Regression or Ridge) is trained. The meta-learner *learns* how much to trust each base model. Using out-of-fold predictions prevents the meta-learner from seeing over-fitted base predictions (data leakage).

	Voting	Stacking
Combination rule	Fixed (majority / average)	Learned by a meta-model
Extra training	None	K-fold base predictions + meta-model
Strength	Simple, robust, fast	Can exploit complementary strengths; usually the most accurate blend
Weakness	Equal trust in all members	More complex; risk of over-fitting the meta-level on small data

5. AdaBoost vs XGBoost (conceptual)

AdaBoost (Adaptive Boosting) maintains a weight w_i for every training sample. In round t a weak learner h_t is fitted to the weighted data, its weighted error ε_t is measured, and it receives a vote weight:

$$\alpha_t = \tfrac{1}{2} \cdot \ln((1 - \varepsilon_t) / \varepsilon_t) \quad w_i \leftarrow w_i \cdot \exp(\alpha_t \cdot 1[h_t(x_i) \neq y_i]) \text{ (then re-normalise)}$$

Misclassified samples therefore get heavier weights, forcing the next learner to concentrate on them. Final prediction: $H(x) = \text{sign}(\Sigma\, \alpha_t \cdot h_t(x))$. (scikit-learn uses the multi-class SAMME variant, and a learning-rate parameter that shrinks α_t; AdaBoost.R2 handles regression.)

XGBoost (eXtreme Gradient Boosting) fits trees to the **gradient** of a loss function and uses a **second-order** (Newton) approximation. Its regularised objective is:

$$Obj = \Sigma_i\, l(y_i, \hat{y}_i) + \Sigma_k\, \Omega(f_k), \quad \Omega(f) = \gamma \cdot T + \tfrac{1}{2} \cdot \lambda \cdot \Sigma\, w_j^2$$

where T is the number of leaves and w_j are leaf weights. Together with shrinkage (learning rate), row/column sub-sampling, built-in missing-value handling and cache-aware/parallel split finding, this makes XGBoost faster and better regularised.

	AdaBoost	XGBoost
Core idea	Re-weight samples; weight learners by accuracy	Fit gradient/Hessian of a differentiable loss
Typical learner	Stumps / depth 1-3 trees	Regularised trees, depth 3-8
Loss	Exponential-type (classification), AdaBoost.R2 (regression)	Any differentiable loss (log-loss, squared error, …)
Regularisation	Learning rate, number of learners	λ (L2), α (L1), γ, min_child_weight, sub-sampling, learning rate, early stopping
Missing values	Must be imputed	Handled natively
Speed	Slower, mostly sequential	Fast (parallel, histogram-based)
Noise sensitivity	High (noisy samples gain weight)	Lower, thanks to regularisation

Glossary

Term	Meaning
Weak learner	A model only slightly better than random guessing (e.g. a decision stump).
Bootstrap	Sampling n rows *with replacement* from n rows.
Shrinkage / learning rate	Factor (0–1] that scales each new learner's contribution to slow down fitting and reduce over-fitting.
Meta-learner	The model that combines base-model predictions in stacking.
ROC-AUC	Probability that a random positive is ranked above a random negative; threshold-independent ranking quality.
RMSE / MAE / R^2	Root-mean-squared error / mean absolute error / proportion of variance explained.